K

GSOC 2026 PROPOSAL -Apache

Apache SkyWalking: BanyanDB Native Data Export/Import Utility

---

1. Personal Information

- **Name:** Jayasankar Tanguturi
- **GitHub:**
- **Email:** jayasankartanguturi@gmail.com
- **University:** SRM AP

---

2. Abstract

BanyanDB from Apache SkyWalking is a high-performance storage engine for observability data, including logs, metrics, and traces. Nevertheless,

itspresent data portability features are restricted to simple CSV exports and backup procedures, which are inadequate for contemporary production settings.
This proposal presents a high-performance, scalable, and extensible export/import utility that supports various formats, including Binary, Parquet, JSON, and CSV, and facilitates smooth data transfer across environments.
To guarantee dependability and performance at scale, the system will make use of streaming, concurrency, and schema evolution techniques.

---

3. Problem Statement

❌ Current Limitations

- There is only CSV export available.
- Ineffective migration between clusters
- Large datasets are not supported.
- No handling of schema compatibility
- limited integration with analytics tools

🚨 Impact

- Debugging workflows that are challenging
- Inadequate scalability
- Low uptake in business use cases

---

4. Goals & Deliverables

Primary Goals

- Construct an import/export system
- Accept a variety of formats
- Assure performance and scalability
- Manage the evolution of schemas

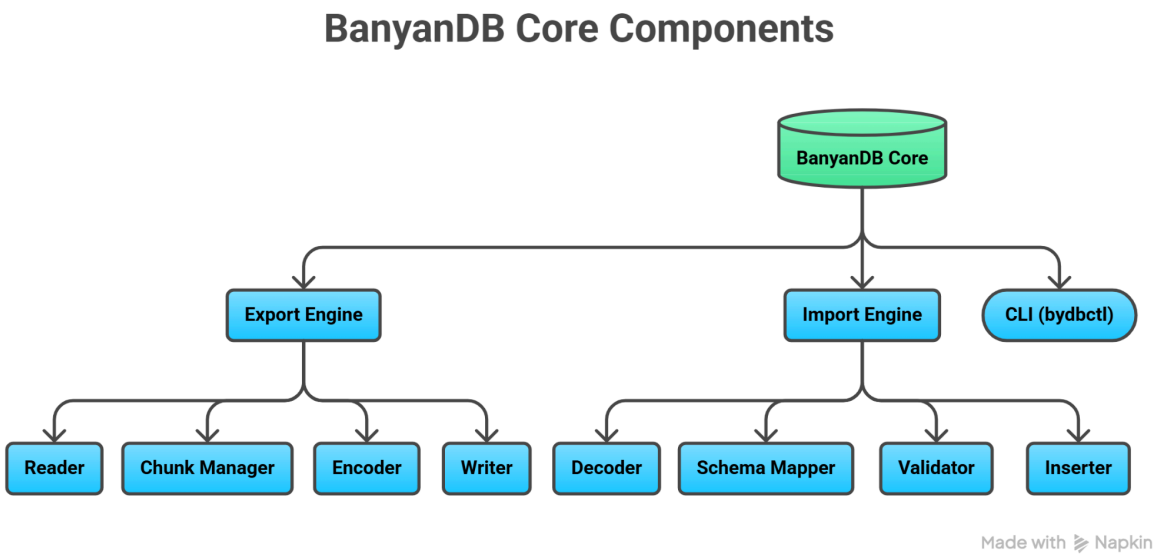
Deliverables

- Binary import/export
- Support for CSV, JSON, and Parquet
- Batch processing combined with streaming
- CLI integration (bydbctl)
- Benchmarks and documentation

---

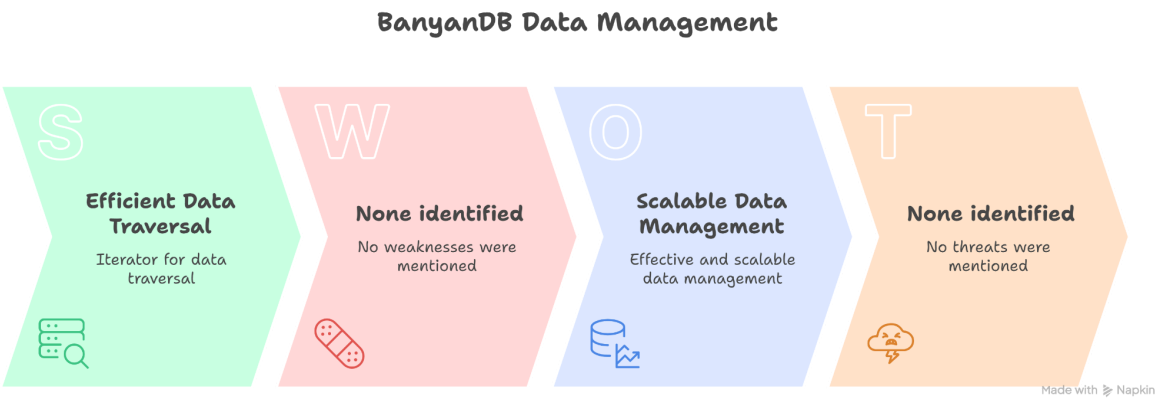
5. System Architecture

5.1 High-Level Architecture

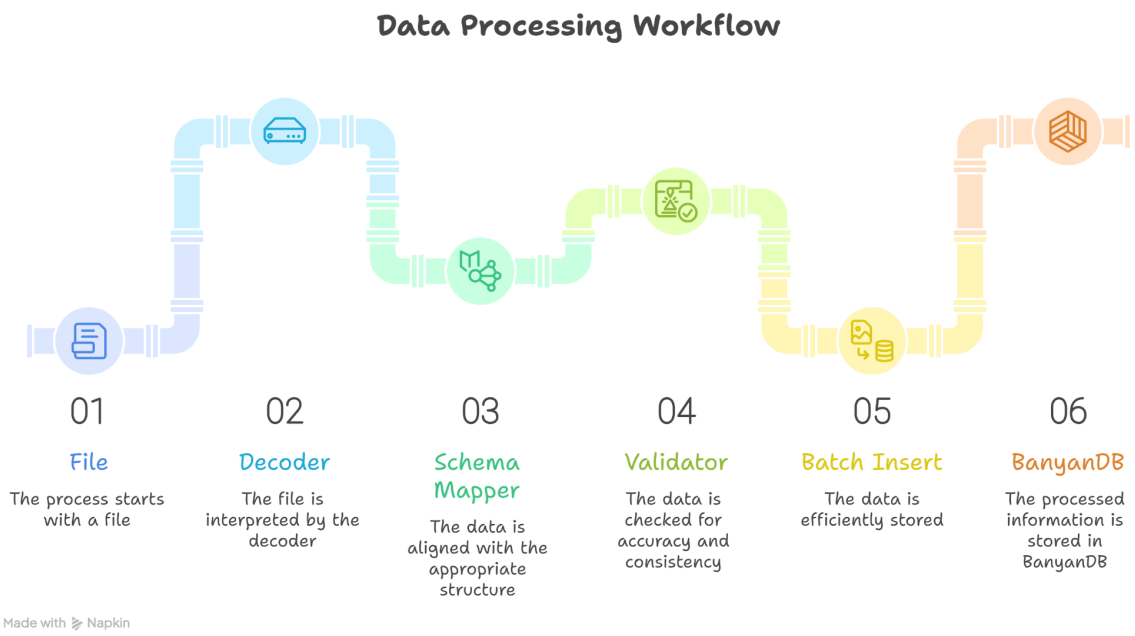


5.2 Data Flow

Export Flow



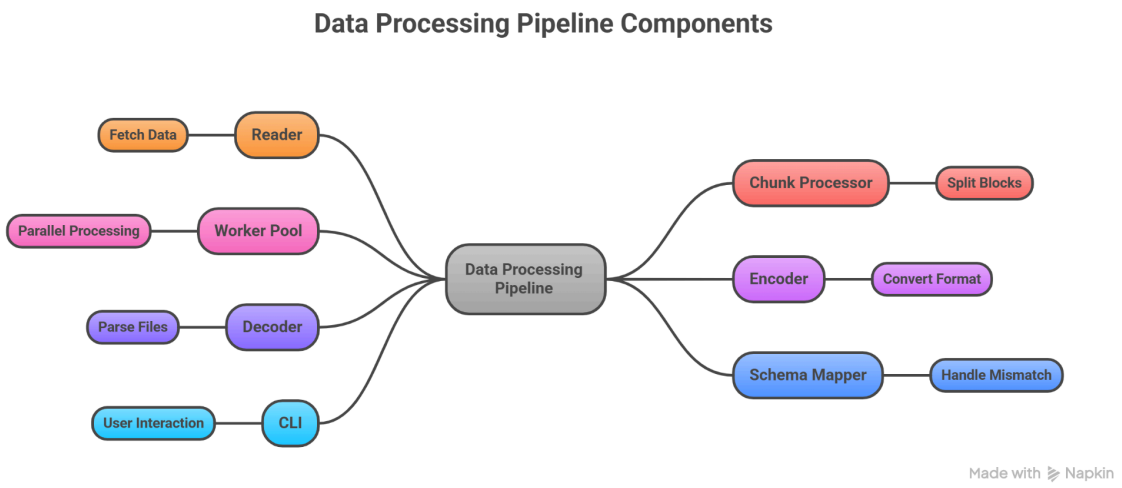
Import Flow



---

6. Technical Design

6.1 Core Components



6.2 Concurrency Model

- Goroutines for parallel execution
- Channel-based communication
- Worker pool pattern

---

7. Performance Strategy

🚀 Optimizations

Parallel Encoding

- Chunks are processed concurrently by the worker pool.
- increases the use of the CPU

Buffered I/O

- For effective disc operations, use bufio.

Streaming

- Process data piecemeal to avoid using up all of the memory.

Columnar Format (Parquet)

- Analytics-focused
- Improved compression

Additional

- Batch inserts
- Snappy/gzip compression
- Using pprof for profiling

8. Data Format Strategy

Format	**Use Case**
Binary	Fast Migration
parquet	Analytics
json	Debugging
csv	Light weight

9. Implementation Timeline

Community Bonding Period (Before Week)

- Learn about the structure of BanyanDB, including its storage layers, APIs, and query model.
- Set up the system for building and developing
- Look into current ways to export data, like CSV and backup tools.
- Talk about design choices with your mentors and make your proposal better.
- Find small problems to start with and send in your first PR (very important)

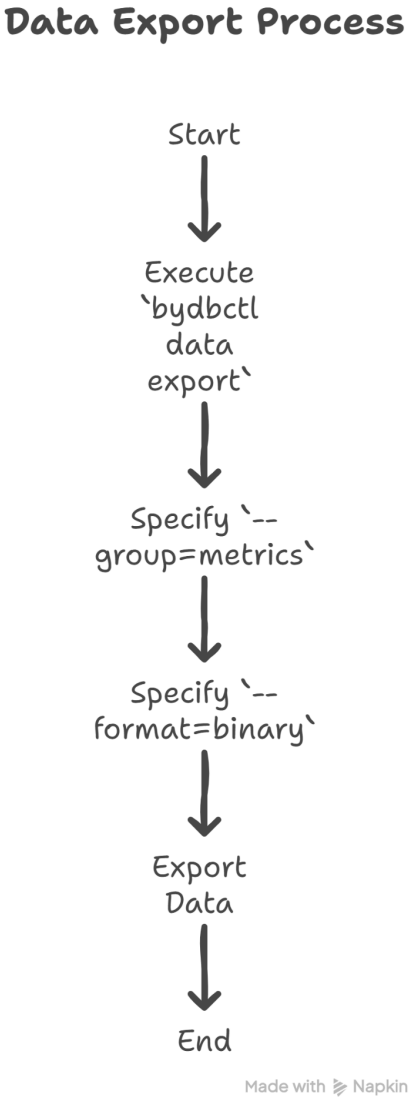
Week 1–2: Deep Research & System Design

- Look at BanyanDB's internal data model, which includes metrics, logs, and traces.
- Look into the iteration APIs and storage engine interfaces.
- Finish the architecture and data flow for export and import
- Create an abstraction layer for design formats (Binary, JSON, CSV, Parquet)
- Make the first structure for the CLI command (bydbctl data export/import)
- Make a technical design document for your mentor to look over.

Milestone: Architecture + design approved

Week 3–4: Core Export Pipeline (MVP)

- Put into action:
- Data reader that uses iterator APIs
- System for processing chunks
- Simple file writer
- Make native binary export (MVP)
- Add a basic command line interface



Milestone: Working export system (binary only)

Week 5–6: Multi-format Support

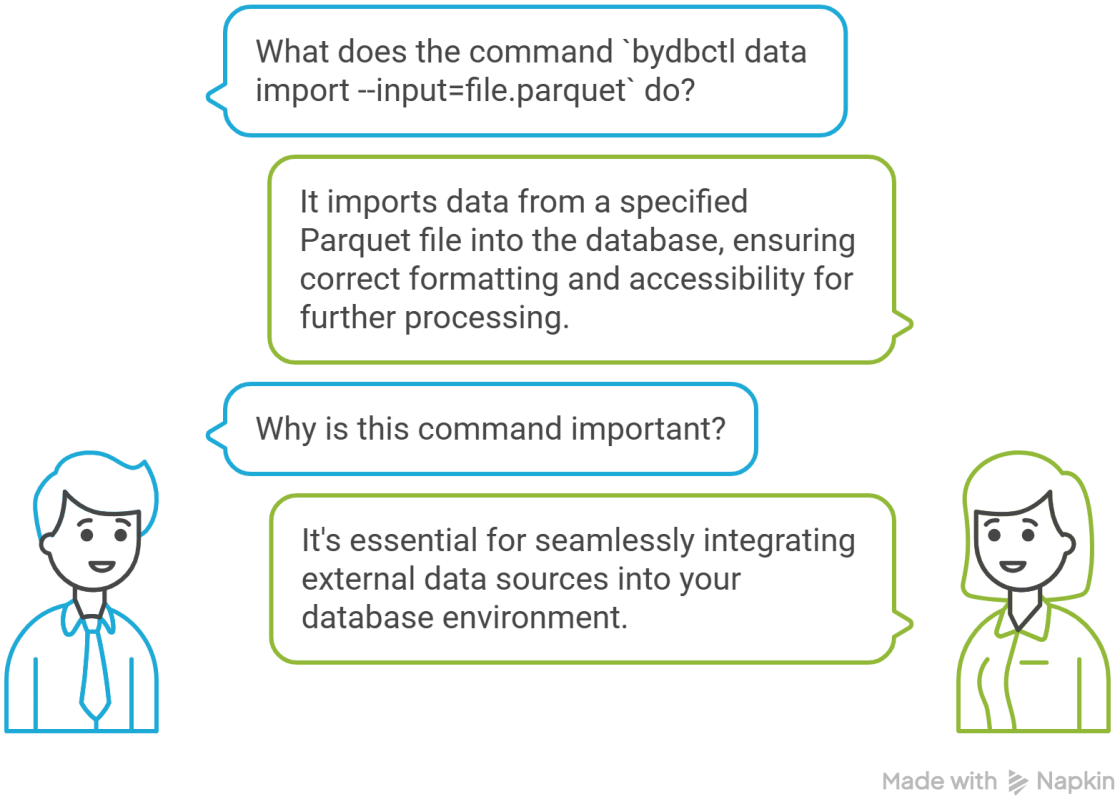
- Use CSV and JSON encoders
- Use Apache Arrow to incorporate Parquet support.
- Create an interface for the format adapter (pluggable design)
- Compare various formats (size, speed).
- Enhance CLI settings (filters, output path)

Milestone: Multi-format export completed

Week 7–8: Import Pipeline Development

- Implement file readers for all formats
- Build decoding layer (JSON, CSV, Parquet, Binary)
- Implement batch insertion into BanyanDB
- Add validation layer (data correctness checks)
- Extend CLI

Milestone: End-to-end export → import working



Week 9–10: Schema Evolution & Compatibility

- Manage:
- Absence of fields
- Extra fields
- Type inconsistencies

- Put the schema mapping layer into practice.
- Include default value handling
- Introduce checks for version compatibility

Milestone: Robust schema handling system

Week 11: Performance Optimization

- Introduce worker pool for parallel processing
- Optimize chunk size dynamically
- Add buffered I/O (bufio)
- Reduce memory footprint (streaming improvements)
- Add optional compression (gzip/snappy)

Milestone: Optimized and scalable system

Week 12: Testing & Benchmarking

- Unit tests for all modules
- Integration tests (full pipeline)
- Load testing with large datasets
- Benchmark:
- Throughput
- Memory usage

- File sizes (per format)
- Fix bugs and edge cases

Milestone: Stable and production-ready build

Week 13: Documentation & Final Submission

- Write user documentation (CLI usage, examples)
- Developer documentation (architecture, modules)
- Add code comments and cleanup
- Prepare demo (sample dataset export/import)
- Final mentor review and improvements

Milestone: Final polished submission

10. Testing Strategy

Unit Testing

- Each module independently tested

Load Testing

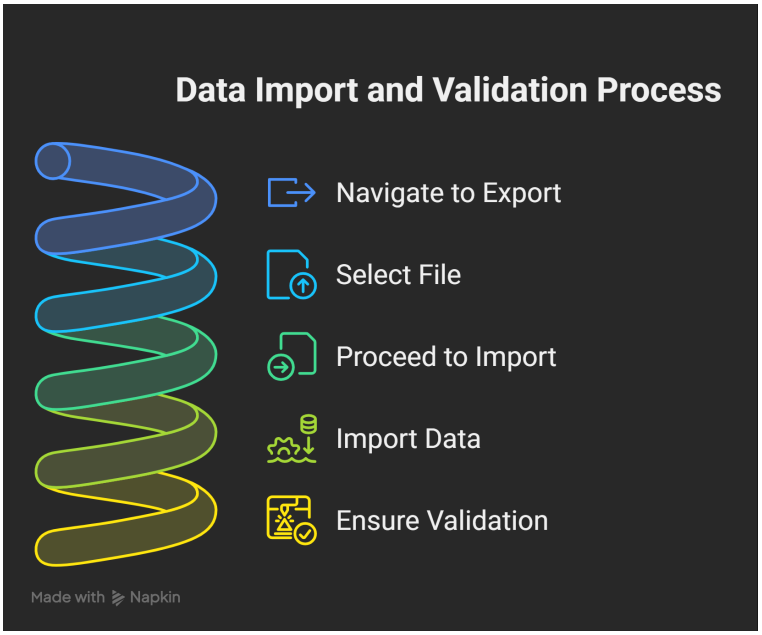
- Large datasets
- Measure performance

Edge Cases

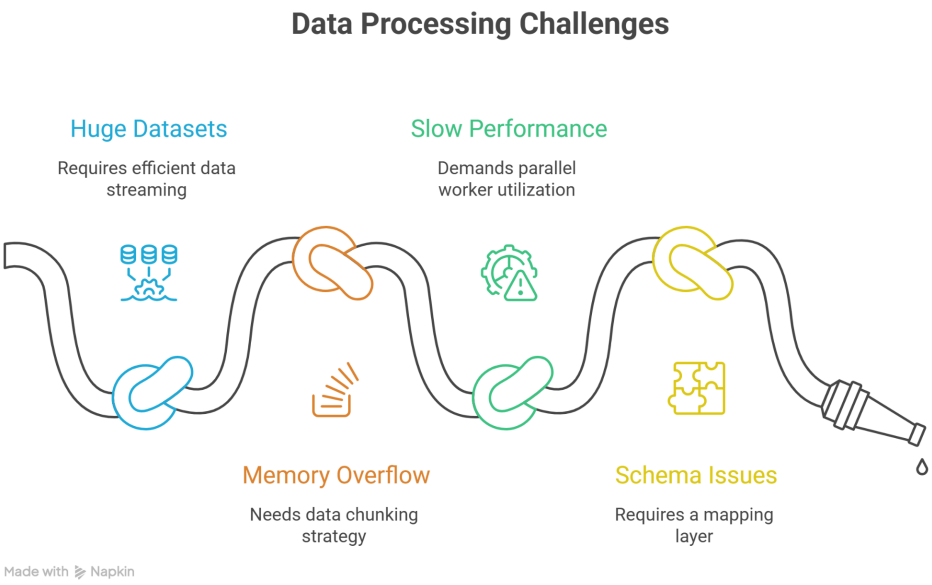
- Schema mismatch

- Corrupted data
- Empty input

Integration testing



11. Risks & Mitigation



12. Code Implementation

12.1 Worker Pool

```go
type Job struct {
    Data interface{}
}

type WorkerPool struct {
    jobs chan Job
}

func NewPool(n int) *WorkerPool {
    p := &WorkerPool{jobs: make(chan Job, 100)}

    for i := 0; i < n; i++ {
        go func() {
            for job := range p.jobs {
                process(job)
            }
        }()
    }
    return p
}
```

12.2 Streaming Export

```go
func StreamExport(records <-chan Record, writer io.Writer) error {
    for r := range records {
        data := encode(r)
        _, err := writer.Write(data)
        if err != nil {
            return err
        }
    }
    return nil
```

12.3 Schema Mapping

```go
func MapSchema(src map[string]interface{}, target []string) map[string]interface{} {
  out := make(map[string]interface{})

  for _, field := range target {
    if val, ok := src[field]; ok {
      out[field] = val
    } else {
      out[field] = nil
    }
  }
  return out
}
```

13. Future Scope

- Gradual export
- Integration of cloud storage
- UI dashboard for real-time streaming

14.Technical skills

- Strong in C, C++, Java, JS
- Good understanding of backend systems
- Passion for distributed systems

- Highly motivated to contribute

15. Conclusion

This project adds a high-performance, scalable data portability system to BanyanDB. It directly enhances Apache SkyWalking's usability, analytics integration, and production readiness.